Auto-generating Quantized DNN Kernels using TVM

Dimitrije Pešić

1 Why Quantization Matters

ResNet50 has approximately 25 million parameters, occupying ∼100 MB stored as float32. Per inference, it executes roughly 4 billion multiply-accumulate (MAC) operations. On edge hardware like Raspberry Pi 4, even a single inference can carry significant latency. Quantization addresses this by representing weights and activations as lower-precision numbers, trading numerical precision for reductions in memory, bandwidth, energy, and compute. It bridges the gap between models we can train and models we can deploy at scale.

Wu et al. (2020) frame int8 as a sweet spot for DNN quantization. We get ∼4× weight compression (verified empirically in this work: the int8 ResNet50 `.params` file is 30 MB versus 191 MB for fp32, including TVM's serialization metadata), proportional activation memory bandwidth reduction, and on hardware with int8 acceleration like Intel VNNI or ARM DOT, up to several-fold compute throughput improvements over fp32. Crucially, ResNet50 at int8 retains accuracy to within ∼0.1% top-1 of the fp32 baseline (Wu et al. 2020), thanks to the asymmetric per-tensor scheme formalized by Jacob et al. (2018) and implemented as the canonical pattern in most modern inference toolchains.

Below int8, the precision/accuracy trade-off degrades sharply. Gholami et al. (2021) conclude that maintaining meaningful accuracy at sub-byte precisions requires either quantization-aware training or specialized network architectures.

The literature thus establishes int8 as the sweet spot: above it, the precision trade-off is essentially free; below it, accuracy degrades sharply without specialized training. This report investigates how TVM (Chen et al. 2018), a deep learning compiler, behaves across this boundary through its `relay.quantize` API. We compile and benchmark ResNet50 at fp32 and int8 on x86, analyze why the expected int8 speedup does not materialize with TVM's defaults, test what happens at int4, int2, and int1, and cross-compile both fp32 and int8 to ARM for aarch64 execution under QEMU emulation. Throughout, we anchor TVM's behavior to its source code rather than treating the framework as a black box.

2 TVM Compilation Pipeline

The workflow has three stages: (1) import the PyTorch model into TVM's intermediate representation (Relay IR), (2) optionally apply quantization passes on the IR, and (3) lower the IR to machine code for a target backend.

2.1 Model Import

ResNet50 is loaded from `torchvision.models` with pretrained `IMAGENET1K_V2` weights. To produce a graph representation suitable for TVM, we trace the model with a dummy input of shape `[1,3,224,224]`:

```
scripted = torch.jit.trace(model, torch.randn(1, 3, 224, 224)).eval()
mod, params = relay.frontend.from_pytorch(
    scripted, [("input0", (1,3,224,224))])
```

This produces two objects from the converted TorchScript graph:

- `mod` – the computation graph in Relay IR
- `params` – the trained weights

2.2 Baseline Compilation (fp32)

We compile the Relay IR to x86 machine code by specifying a target. The target tells TVM which instruction set to emit, in this case LLVM's x86 backend. For GPU compilation, this would be replaced with `cuda`.

The build runs inside `PassContext(opt_level=3)`, which enables TVM's optimization passes: operator fusion, constant folding, and layout transformation. The last is why TVM's logs show `conv2d_NCHWc` rather than `conv2d` – TVM rewrites the data layout from NCHW to a tiled format optimized for the x86 cache hierarchy.

```
target = tvm.target.Target("llvm", host="llvm")
with tvm.transform.PassContext(opt_level=3):
    lib = relay.build(mod, target=target, params=params)
```

The result is a compiled module loaded into a `GraphModule` executor. We feed it a random fp32 input and benchmark over 100 runs.

```
m = graph_executor.GraphModule(lib["default"](dev))
m.set_input("input0", tvm.nd.array(img))
timing = m.benchmark(dev, number=100)
```

2.3 Int8 Quantization

TVM applies post-training quantization directly on the Relay IR – no retraining or calibration dataset is needed. The `quantize.qconfig` context manager controls all quantization parameters:

```
with quantize.qconfig(
    nbit_input=8, nbit_weight=8, nbit_activation=32,
    dtype_input="int8", dtype_weight="int8",
    dtype_activation="int32",
    calibrate_mode="global_scale", global_scale=8.0,
    skip_dense_layer=True, skip_conv_layers=[0],
):
    mod_int8 = quantize.quantize(mod, params=params)
```

`nbit_input` and `nbit_weight` control quantization precision – the parameter we vary later when testing sub-byte widths. The default values for every key here are defined in `QConfig._node_defaults` in `python/tvm/relay/quantize/quantize.py`, with a matching C++ definition in `include/tvm/relay/quantize/quantize.h`. This is why TVM "knows" to quantize to int8 even when `dtype_input` is not explicitly set: it is a baked-in framework default.

`calibrate_mode="global_scale"` uses a fixed scaling factor rather than running calibration data through the network; this is acceptable since we are benchmarking throughput, not accuracy. `skip_conv_layers=[0]` leaves the first convolution in fp32, a common practice since the input layer has only 3 channels and quantizing it yields minimal savings. `skip_dense_layer=True` keeps the final classifier layer in fp32.

We verify that quantization took effect by printing the IR: `mod` shows float32 operations throughout, while `mod_int8` shows int8 casts and quantized convolutions. This confirms that `quantize.quantize` transformed the computation graph rather than merely annotating it. Internally, this transformation is implemented in three phases – *Annotate*, *Calibrate*, and *Realize*

– with the operator-level rewriting performed in `src/relay/quantize/realize.cc` via per-op rules (`Conv2dRealize`, `AddRealize`, `MulRealize`, `ClipRealize`, etc.).

The quantized module is then compiled and benchmarked identically to the fp32 baseline:

```
with tvm.transform.PassContext(opt_level=3):
    lib_int8 = relay.build(mod_int8, target=target,
                           params=params)
```

3 Results

3.1 Benchmarking: fp32 vs int8

We benchmarked both models on x86 CPU (LLVM backend) over 100 inference runs each. All measurements were collected under the same conditions in a single session on a WSL2 host. The results:

Config	Mean (ms)	Median (ms)	Std (ms)
float32	88.04	87.98	0.43
int8 (signed)	228.55	228.00	1.46

We expected int8 to be at least as fast as fp32, and potentially several-fold faster on hardware with int8 acceleration. Instead, the int8 model is 2.6× slower than fp32. Two factors explain this.

Lack of autotuning. TVM prints the warning "One or more operators have not been tuned. Please tune your model for better performance." Without autotuning, TVM falls back to generic schedules for every operator. TVM's autotuners (AutoTVM, Ansor) would search for optimal loop tiling, vectorization, and parallelization parameters for each operator shape on the target hardware. Without this step, the default schedules are naive and far from optimal.

Requantization overhead. The int8 computation graph is significantly more complex than fp32. Examining the `te_compiler` dispatch logs and the post-Realize IR, each convolution block in fp32 consists of three operations:

```
conv2d_NCHWc -> add -> relu
```

In int8, the same logical convolution becomes a chain of 13 operations:

```
conv2d_NCHWc -> add -> right_shift -> clip -> cast ->
cast -> multiply -> add -> relu -> add ->
right_shift -> clip -> cast
```

The extra operations (`right_shift`, `clip`, `cast`, `multiply`) implement the dequantization arithmetic: after computing in the int32 accumulator, the result must be rescaled (via a fixed-point multiplier or shift, depending on which path `MulAndDiv` in `realize.cc` selects), clipped to the int8 range, and cast back. This 13-op chain replaces every convolution in the network, and additional requantization ops appear around the residual additions that join the ResNet bottleneck blocks.

Dispatch hypothesis. For int8 to be faster than fp32, TVM would need to dispatch the convolutions to a hardware-accelerated kernel. Tracing the dispatch logic, we find that `conv2d_NCHWc_strategy_cpu` (in `python/tvm/relay/op/strategy/x86.py`) chooses between two kernel implementations based on the helper `is_int8_hw_support` (defined in `python/tvm/topi/x86/`

conv2d_int8.py). That helper requires data_dtype == "uint8" *and* kernel_dtype == "int8" – an asymmetric pairing inherited from Intel's VPDPBUSD intrinsic (visible in python/tvm/-topi/x86/tensor_intrin.py), which operates on byte-*unsigned* × *signed*-byte → int32. TVM's default qconfig produces *signed* int8 for both data and weights, so the dispatch falls through to the generic conv2d_NCHWc.x86 kernel rather than the VNNI-accelerated conv2d_NCHWc_int8.x86 path. The int8 graph thus pays the requantization overhead without gaining any compute throughput advantage. This dispatch hypothesis is tested empirically in Section 4.2, where the same int8 graph is cross-compiled to an ARM target with a different operator strategy.

In a separate experiment we tested the VNNI-preferred configuration directly (dtype_-input="uint8", dtype_weight="int8"). In that session the uint8/int8 configuration measured 330.19 ms versus 365.64 ms for signed int8/int8 – marginally faster, but still slower than fp32. This is consistent with the absence of autotuning being the dominant factor on x86: even on the dispatch-preferred path, untuned schedules cannot beat the simpler fp32 graph.

3.2 Sub-byte Quantization: int4, int2, int1

The qconfig API accepts arbitrary bit widths through nbit_input and nbit_weight. We tested int4, int2, and int1 to determine which precisions TVM can handle end-to-end.

3.2.1 int4 (4-bit)

Quantization succeeded: quantize.quantize() returned a valid Relay module with int4 dtypes present in the IR (461 occurrences of "int4" in the IR text). However, relay.build() failed during LLVM code generation. The error originates in src/target/llvm/codegen_llvm.cc:1716 (CodeGenLLVM::BufferAccessHelper), where the check value_dtype.bits() $\geq$ 8 fails. This encodes the architectural assumption that buffer access dtypes must be at least byte-sized. TVM's LLVM codegen has no support for sub-byte integer types.

3.2.2 int2 (2-bit)

The process entered a non-terminating loop during the quantization step itself, before relay.build was even invoked. Using Python's faulthandler with periodic stack dumps (every 30 seconds), we located the hang inside prerequisite_optimize (python/tvm/relay/quantize/quantize.py:329), which runs standard Relay optimization passes (SimplifyInference, FoldConstant, FoldScaleAxis, CanonicalizeOps) on the float32 module before quantization. The process consumed ∼97% CPU on a single core with stable memory, and eventually segfaulted. The int2 dtype is not yet present in the IR at this stage (these passes run on the float32 module), but the active qconfig influences pass behavior through shared state.

3.2.3 int1 (1-bit)

Quantization succeeded, but relay.build() raised an InternalError with a clear stack trace. The failure occurs in the SimplifyExpr optimization pass (src/relay/transforms/simplify_-expr.cc:170). The pass calls CheckDataTypeMaxMinValue to validate a clip op's bounds against its target dtype. TVM treats DataType::Int(1) as effectively boolean with range $[0, 1]$, so constructing IntImm(int1, -1) for the quantization-generated clip's lower bound fails the range assertion:

```
Check failed: (value == 0 || value == 1) is false:
ValueError: -1 exceeds range of int1
```

The error path is: relay.build → OptimizeImpl → SimplifyExpr → SimplifyClipAnd-ConsecutiveCast::Callback → CheckDataTypeMaxMinValue → tvm::min_value(int1) → In-tImm(int1, -1) → assertion failure.

3.2.4 Summary of Sub-byte Failures

Precision	Quantize	Build	Failure Location
int8	OK	OK	(works)
int4	OK	FAIL	LLVM codegen: `bits` ≥ 8
int2	HANG/CRASH	N/A	`prerequisite_optimize` loop
int1	OK	FAIL	`SimplifyExpr`: int1 range

These three failure modes confirm that TVM's quantization framework (Annotate → Calibrate → Realize) is dtype-agnostic and accepts arbitrary `DataType::Int(N)` values without validation. The breakdown happens downstream when optimization or codegen passes encounter the resulting IR. Each pass imposes its own dtype-specific assumptions, and there is no unified validation layer. The framework fails at varied stages, with varied symptoms ranging from clean exceptions to non-terminating loops, depending on which assumption is violated first.

4 Cross-Compilation for ARM

4.1 Compiling for Raspberry Pi

To target the Raspberry Pi 4 (Cortex-A72), we change the compilation target string to specify the ARM instruction set:

```
target = tvm.target.Target(
    "llvm␣-device=arm_cpu␣"
    "-mtriple=aarch64-linux-gnu␣"
    "-mcpu=cortex-a72"
)
```

The target triple `aarch64-linux-gnu` instructs LLVM to emit 64-bit ARM code for Linux, and `mcpu=cortex-a72` enables microarchitecture-specific optimizations for the Pi 4's CPU. Cortex-A72 is an ARMv8-A core with standard NEON SIMD; it does not include the dot-product instructions introduced in ARMv8.2+.

Since we are cross-compiling (building ARM binaries on an x86 host), we use `aarch64-linux-gnu-gcc` as the cross-compiler when emitting the shared library:

```
from tvm.contrib import cc as _cc

def cross_cc(output, files, options=None,
             cc="aarch64-linux-gnu-gcc"):
    return _cc.create_shared(output, files,
                             options, cc=cc)

lib.export_library("resnet50_fp32_arm.so",
                   fcompile=cross_cc)
```

Both the fp32 and int8 models are exported as three files each:

- `.so` – the compiled shared library containing operator kernels
- `.json` – the graph structure (execution order, operator names, shapes)
- `.params` – the serialized weights

This produces six files total. The aarch64 ELF format of each `.so` was verified with the `file` utility:

```
$ file resnet50_fp32_arm.so
resnet50_fp32_arm.so: ELF 64-bit LSB shared object,
ARM aarch64, version 1 (SYSV), dynamically linked,
with debug_info, not stripped
```

The int8 `.params` file is 30 MB versus 191 MB for fp32, confirming the storage benefit of int8 quantization independent of any compute-side speedup.

4.2 QEMU Emulation

To execute the cross-compiled binaries without physical Raspberry Pi hardware, we set up an aarch64 Ubuntu Server 22.04 virtual machine under `qemu-system-aarch64` (full-system emulation), configured to emulate a Cortex-A72 machine with 4 GB RAM and 4 vCPUs. After booting and confirming `uname -m` returns `aarch64`, we installed Python, NumPy, and the TVM v0.15.0 source inside the VM, and built the runtime-only TVM target (`make runtime` with `USE_LLVM=OFF`). The six exported files were copied into the VM via `scp` over the forwarded SSH port.

A small benchmark script inside the VM loads each artifact triple via `tvm.runtime.load_module`, `graph_executor.create`, and `load_params`, runs a warmup iteration, and measures latency over a small number of inferences (kept low because of QEMU overhead).

Config	Mean (ms)	Median (ms)
float32 (aarch64, QEMU)	9550.5	9603.8
int8 (aarch64, QEMU)	4619.2	4693.5

Table 1: ResNet50 inference latency under QEMU full-system emulation of Cortex-A72. Each measurement is the mean of three repeats of two inferences each, preceded by a single warmup run. QEMU adds roughly 30–50× overhead relative to native execution, so absolute values do *not* represent real Raspberry Pi performance; the ratio between fp32 and int8, however, is informative because both configurations incur the same emulation cost.

Two observations stand out. First, both models produced sensible classification outputs (fp32 `argmax`=21, int8 `argmax`=812, both with output ranges similar to the fp32 x86 baseline), confirming that the cross-compiled aarch64 binaries are functionally correct. Second, and more interestingly, under aarch64 emulation the int8 model runs 2.07× ***faster*** than fp32 – the opposite of what we measured on x86, where int8 was 2.6× *slower*.

This inversion is direct empirical support for the dispatch hypothesis developed in Section 3.1. On x86, TVM's optimized int8 conv kernel requires a `uint8 × int8` dtype pairing dictated by the asymmetric `VPDPBUSD` intrinsic, which the default qconfig does not produce, so dispatch falls through to a generic kernel. On aarch64, the int8 kernel paths in TVM's ARM operator strategy (under `python/tvm/topi/arm_cpu/`) do not impose the same asymmetric requirement, and signed int8 inputs and weights flow through int8-specific schedules that benefit both from reduced data movement and from NEON's vectorized integer multiply-accumulate. The expected quantization speedup materializes – even under QEMU's substantial emulation overhead. The bottleneck on x86 was therefore the specific interaction between TVM's default qconfig and the platform's dispatch logic, not int8 quantization itself.

5 Conclusion

We compiled and benchmarked ResNet50 at fp32 and int8 using TVM's `relay.quantize` API on x86 CPU. The int8 model was 2.6× slower than fp32 (228.55 ms vs 88.04 ms). Source-code

analysis attributes this to two compounding causes: (i) TVM dispatches convolutions to a generic, untuned `conv2d_NCHWc.x86` kernel because `is_int8_hw_support` requires a uint8/int8 dtype pairing that the default qconfig does not produce, and (ii) the int8 graph carries substantial requantization overhead (13 ops per convolution block vs 3 in fp32) implemented via the per-op rewriting rules in `src/relay/quantize/realize.cc`.

The cross-platform experiment closes the loop on this analysis empirically. Cross-compiling the same int8 graph for aarch64 and executing under QEMU emulation of Cortex-A72, the int8 model ran 2.07× *faster* than fp32 – the inverse of the x86 result. The expected quantization speedup is therefore not blocked by anything intrinsic to int8 quantization, but by the specific interaction between TVM's default qconfig (signed int8 for both data and weights) and the x86 dispatch logic governed by Intel's asymmetric VNNI intrinsic. On a target whose operator strategy accepts signed int8 directly, the expected speedup materializes even under heavy emulation overhead.

Sub-byte quantization (int4, int2, int1) revealed that TVM's quantization pipeline is dtype-agnostic at the Relay level but lacks end-to-end support for sub-byte types. Each bit width fails at a different stage of the pipeline: int4 at LLVM codegen (sub-byte buffer access), int2 at pre-quantization optimization (non-terminating loop ending in segfault), and int1 at post-quantization simplification (int1 range assertion). These failures are consistent with TVM's source code, which assumes a small set of well-known dtype ranges (int8, int16, int32, float32) throughout its optimization and codegen infrastructure.

We successfully cross-compiled both fp32 and int8 models for the Raspberry Pi 4 (Cortex-A72) using TVM's cross-compilation support with `aarch64-linux-gnu-gcc`, producing deployable `.so`/`.json`/`.params` artifacts verified as aarch64 ELF objects. The ∼4× weight compression from int8 quantization (30 MB vs 191 MB) remains valuable for deployment on memory-constrained devices regardless of which dispatch path the inference workload takes. For production use on x86, the critical missing steps are autotuning (AutoTVM or Ansor to find optimized schedules for each operator shape) and adopting a uint8/int8 dtype scheme that hits the VNNI-accelerated dispatch path; on newer ARM cores, an analogous dot-product dispatch path becomes available with ARMv8.2+ targets.

References

[1] Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., & Krishnamurthy, A. (2018). TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of OSDI '18*.

[2] Wu, H., Judd, P., Zhang, X., Isaev, M., & Micikevicius, P. (2020). Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. *arXiv:2004.09602*.

[3] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2021). A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv:2103.13630*.

[4] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of CVPR '18*, pp. 2704–2713.